

Smart Rental Tracking System

Final architecture, work division, and execution map — Phases 03 through 11

Source of truth: `.ai/TEAM-EXECUTION-PLAN.md` in the repository. This PDF is a study copy generated from that file — if they ever disagree, the markdown file wins.

What this document is

A single reference so all four team members know exactly what the system does, how the architecture works, who owns which phase and files, where the real dependencies are, how to work in parallel without collisions, how the presentation is divided, and how to defend their part under panel questioning.

Project status at a glance

Phase	Status	What it covers
00 — Foundation	VERIFIED	Repo, stack, AI-agent operating system
01 — Data model	VERIFIED	PostgreSQL schema, migrations
02 — Synthetic data	VERIFIED	Official 7-row sample + realistic seeded history
03 — Core APIs	Next	Check-out / check-in / usage logging
04-06 — Analytics	Planned	Alerts, anomalies, forecasting
07 — Recommendations	Planned	Signal → reason → action → impact
08-09 — UI	Planned	Asset Dashboard, Control Tower
10-11 — Integration & demo	Planned	End-to-end verification, panel-defense prep

Problem & core user journey

The problem: Caterpillar wants a credible operational control tower for rental equipment — track each asset from checkout to return, show whether it's used well, catch what's going wrong, predict what's needed next, and tell the operator what to *do* about it. A dashboard that only reports scores badly here.

The one user: the rental/fleet operations dispatcher deciding what to check out, what to chase down, what to reassign.

Core asset lifecycle



Every status change on this line records **who / what / where / when**.

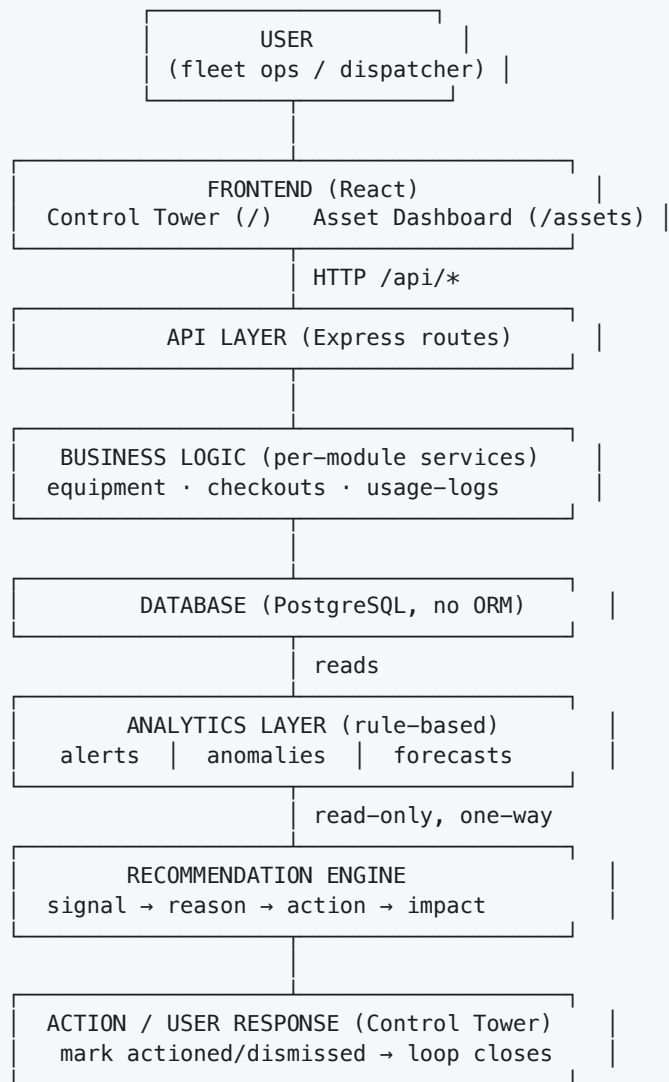
The demo narrative the product must produce live



Differentiation strategy

Every team gets the same brief and the same AI tools; most will ship a table + a couple of charts + a checkout form + a bolted-on chatbot. Citadel differentiates by doing the **Action Queue** properly: every alert/anomaly/forecast collapses into one ranked signal → reason → recommended action → expected impact item, backed by cited thresholds, not vibes.

High-level system architecture



Two independent apps (client/server) over a versionless `/api` HTTP contract. Analytics and recommendations are plain service functions reading the same Postgres tables — no ML pipeline, no message queue, no new infrastructure.

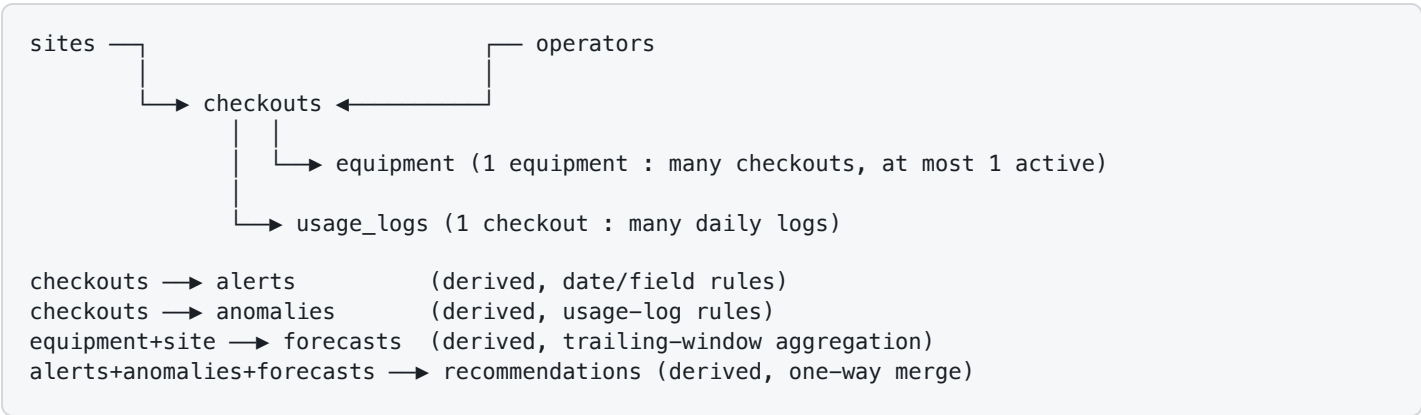
End-to-end data flow



Asset / user lifecycle mapped to phases

Step	CHECK OUT	ASSIGN&TRACK	LOG USAGE	ANALYZE	ALERT/ANOMALY	RECOMMEND	ACT	CHECK IN
Phase	03	03	03	05	04/05	07	09	03

Database / entity relationship



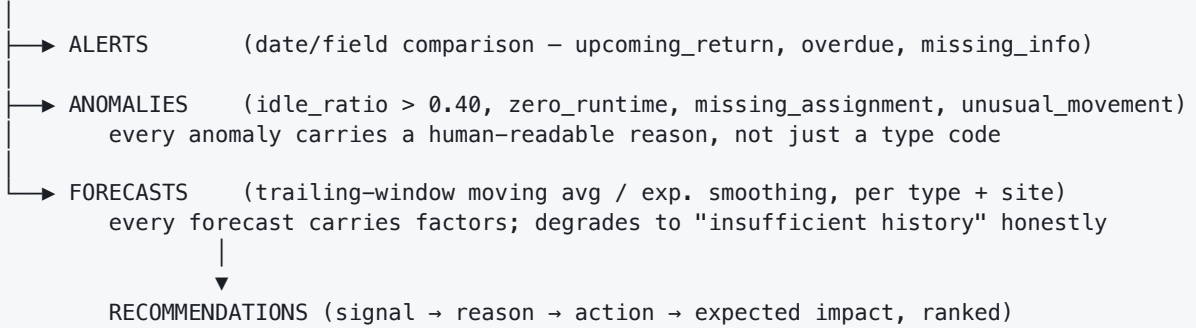
Design decision worth defending: `checkouts.site_id` and `checkouts.operator_id` are nullable by design — this is what makes `MISSING_ASSIGNMENT` a real, representable database state, matching Caterpillar's own official sample rows (`EQX1002/EQX1007`) exactly. See `.ai/DECISIONS.md`.

Current seeded data (Phase 02, verified)

Table	Rows
sites	6
operators	13
equipment	17 (7 official + 5 volume + 5 live-active)
checkouts	22
usage_logs	192

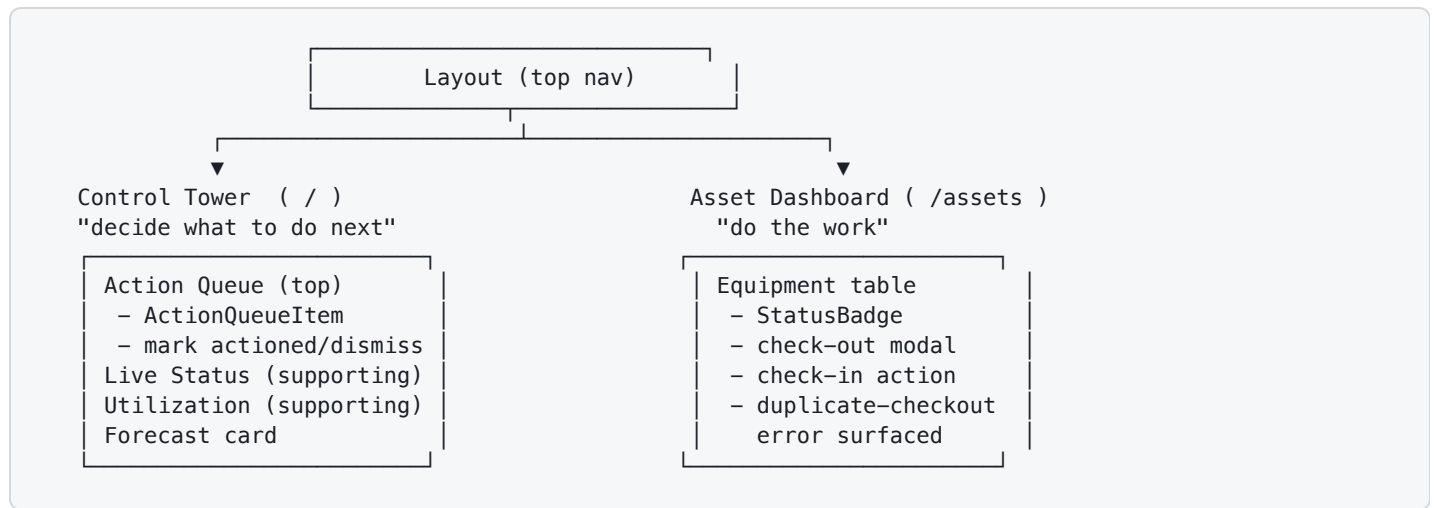
Analytics / AI pipeline

RAW OPERATIONAL DATA (checkouts, usage_logs)



Thresholds are cited, not invented: 0.40 idle-ratio threshold and the 65-75% healthy utilization band come from `.ai/RESEARCH.md` (R-002), already validated once against the real seeded 17-row dataset (10 flagged, 7 clean, no boundary ambiguity — `.ai/DECISIONS.md`). Forecasting method (moving average vs. exponential smoothing) is chosen in Phase 06 against real trailing history, not assumed up front — deliberately rule-based, never a trained model, because 7-asset-scale data cannot honestly support one.

Frontend information architecture



Two screens for MVP — information hierarchy is **Attention** → **Explanation** → **Action**, not chart-first. Color carries meaning only (neutral/amber/red), no decorative gradients or chatbot panel. See `.ai/DESIGN.md`.

4-person team ownership

Person	Primary ownership	Phases	Decision to defend	Cross-training task	Presentation section
Ayush	Foundation, data model, synthetic data, integration	00, 01, 02, drives 10	Why site_id/operator_id are nullable	Builds StatusBadge component (frontend)	Problem, product & foundation
Astik	Core backend APIs, Asset Dashboard UI	03, 08	Duplicate-checkout prevention at the DB level	Implements UNUSUAL_MOVEMENT anomaly rule	How it works
Eklavya	Alerts, anomalies, forecasting	04, 05, 06	Trailing-window forecast vs. trained model	Writes Phase 03 edge-case tests first	Intelligence
Souharda	Recommendations, Control Tower UI	07, 09	One-way read dependency (recs never write back)	Implements usage-logs validation schema	Action & outcome

Phases 10 (integration) and 11 (demo) are whole-team responsibilities — Ayush drives the walkthrough, everyone fixes issues in their own area and rehearses together.

Balance rationale

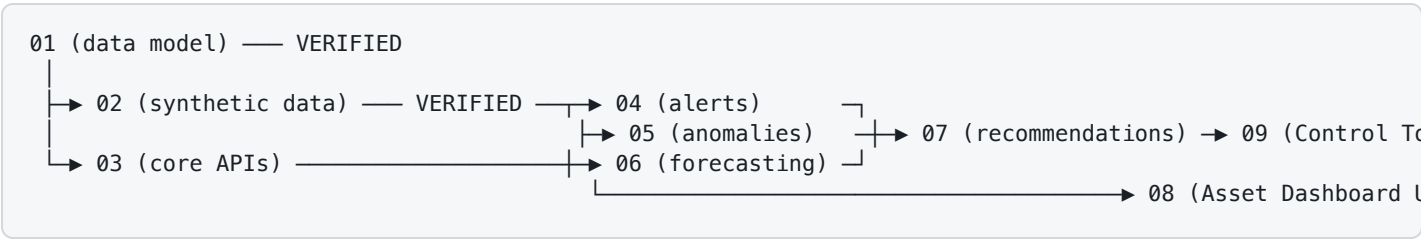
Business Impact (25%) + Innovation (25%) + AI & Analytics (15%) — 65% of the judging weight — sits in Phases 04-07, which is why analytics gets two people (Eklavya building the signals, Souharda building what they become) instead of one, and why every person's cross-training task deliberately touches the layer next to theirs, not a random unrelated task.

Phase 03-11 responsibility map

Phase	Task	Owner	Depends on	Verification
03	equipment / checkouts / usage-logs modules	Astik	01	supertest: duplicate checkout, double check-in, orphan log
04	alerts engine	Eklavya	02, 03	seeded overdue/upcoming-return appear correctly
05	anomaly detection	Eklavya (+Astik: unusual_movement)	02, 03	EQX1002/EQX1007 flagged, EQX1003/EQX1005 clean
06	demand forecasting	Eklavya	02	1 real forecast + 1 insufficient-history case
07	recommendations & action queue	Souharda	04, 05, 06	mapping + ranking + actioned/dismissed persistence
08	Asset Dashboard UI	Astik	03	manual click-through incl. duplicate-checkout error
09	Control Tower UI	Souharda	07 (contract can start early)	manual click-through incl. empty/insufficient states
10	integration, testing, polish	Ayush drives, all fix	03-09	full walkthrough clean, twice
11	demo & panel-defense prep	All four	10	2 clean rehearsals on the real machine

Parallel-safe today (once 03 lands): 04, 05, 06, 08 — zero file collisions between them.

Dependency / parallel execution map



File ownership (avoids merge conflicts)

Path	Owner
server/db/migrations/, server/db/seed.js	Ayush (frozen post-Phase 02 — flag before touching)
server/src/modules/equipment/, checkouts/, usage-logs/	Astik
server/src/modules/alerts/, anomalies/, forecasts/	Eklavya
server/src/modules/recommendations/	Souharda
client/src/pages/AssetDashboard.jsx + api/equipment.js, checkouts.js	Astik
client/src/pages/ControlTower.jsx + api/recommendations.js	Souharda
server/src/routes/index.js, client/src/App.jsx (shared)	One line per module — pull before adding, never delete another's line

Presentation division

#	Person	Section	Content
1	Ayush	Problem, product & foundation	The problem in plain language, the one user persona, why this architecture/database, the data model and why nullable-assignment matters for what's shown later
2	Astik	How it works	System architecture live, the checkout/assign/track/log/check-in workflow, why layered modules + no ORM, how duplicate checkouts are actually prevented
3	Eklavya	Intelligence	Data → alerts → anomalies → forecasting pipeline, cited thresholds, a real flagged anomaly with its reason, a forecast with factors, the honest fallback shown live
4	Souharda	Action & outcome	The Action Queue turning three signal types into one ranked list, marking an item actioned live, the labeled expected-impact figure, business-impact framing

Each presenter follows: **WHAT** → **WHY** → **HOW** → **ALTERNATIVE CONSIDERED** → **TRADEOFF** → **RESULT** — never "I made the frontend."

Team workflow & git checkpoints

1. Pull latest main before starting; claim a task via its phase file / issue.
2. Check the dependency map (page 10) before starting.
3. Work on a focused branch per phase; keep changes scoped to owned files.
4. Verify before claiming completion — code existing ≠ done (`.ai/QUALITY.md`).
5. Merge to main same-day; tag a checkpoint once a phase gate passes.
6. Never force-push to main; never overwrite unfamiliar changes without asking.
7. Update STATE.md/HANDOFF.md immediately after merging (Ayush coordinates this).

Panel defense cheat sheet

Each owner fills in their real answer from their own phase file / DECISIONS.md / RESEARCH.md — not a script written by someone else, because a memorized answer fails under a follow-up.

1. What did you personally build?
2. Why this way — the reasoning, not just the description?
3. How does it actually work, mechanically?
4. What alternative did you consider and reject, and why?
5. What can go wrong with it?
6. How did you test it — what did you actually run?
7. Performance consideration, if any?
8. Security consideration, if any?
9. What tradeoff did you knowingly accept?
10. How does this connect to the rest of the system?

Cross-team questions nobody may dodge

Asked of	Question	Real answer traces to
Astik	Why does the Control Tower rank items this way?	Souharda's Phase 07 ranking logic
Souharda	Why is checkouts.site_id nullable?	Ayush's Phase 01 decision (DECISIONS.md)
Eklavya	How does the UI show "insufficient history"?	Souharda's Phase 09 task 09.8 / DESIGN.md
Ayush	Why not fabricate a forecast number?	Eklavya's Phase 06 REQ-019 fallback

Final readiness checklist

Planning artifact

- Architecture matches the problem
- Every requirement has a phase + owner
- Database supports the workflows
- APIs support the frontend (page 4 contract)
- Analytics has required inputs

Team readiness

- All 4 people have meaningful, balanced work
- Presentation is balanced (page 11)
- Ownership overlaps identified (page 10)
- Parallel work is safe today
- Panel-defense responsibilities clear

Before Phase 11 is declared complete

- All required phases 03-11 completed and verified, not just coded
- Core user journey (checkout → check-in) works end-to-end
- Frontend, backend, database, analytics all independently verified
- Anomalies, forecasting, recommendations each produce a real, explainable result against seeded data
- Security baseline checked (no secrets committed, parameterized queries, input validation)
- Application runs from a clean setup; demo path rehearsed twice on the real machine
- Documentation (.ai/*) reflects what was actually built
- Known issues explicitly documented — none silently ignored
- Final git state clean, stable checkpoint pushed and verified on remote

Full detail and rationale for every item above: `.ai/TEAM-EXECUTION-PLAN.md`.